

De la voz al tópico: anatomía del pipeline, paso a paso

Seguimos una sola frase real desde el audio hasta el perfil del entrenador, mostrando qué le ocurre al dato en cada etapa y por qué.

Proyecto **Voz** · Procesamiento de Lenguaje Natural · Ingeniería en IA · Universidad de Palermo · Junio 2026

El objetivo de esta nota no es repetir la teoría, sino **mostrarla en acción**. Tomamos una frase real de una conferencia de Marcelo Gallardo y la hacemos atravesar todo el pipeline. En cada paso vemos el dato antes y después, y explicamos la intuición de la técnica. Así se entiende por qué el topic modeling puede "descubrir de qué habla" un entrenador sin que nadie se lo diga de antemano.

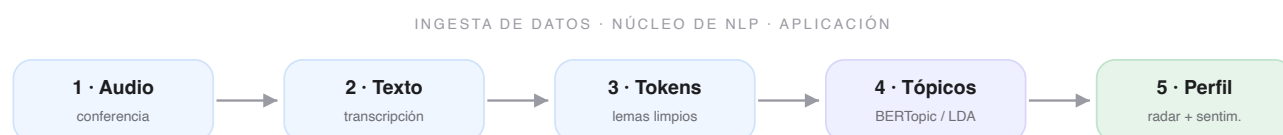


Figura 1. El pipeline completo. Los pasos 1–2 son ingesta de datos; el paso 3–4 es el aporte de NLP (topic modeling); el paso 5 es la capa de aplicación. La frase de ejemplo recorrerá estas cinco estaciones.

1 Del audio al texto

Reconocimiento automático del habla (ASR) — un paso de ingesta, no de NLP

Una conferencia es audio. Para analizarla necesitamos texto. `yt-dlp` descarga el audio del video y `faster-whisper` —un modelo de reconocimiento del habla— lo transcribe en español. Es la misma familia de tecnología que usa el dictado del teléfono. No es nuestro aporte de NLP: es simplemente cómo conseguimos el dato.

NUESTRO EJEMPLO

"Estoy muy contento con el equipo, los jugadores entendieron el partido y dieron todo en la cancha."

Una frase como tantas que dice un DT tras un buen partido. A partir de acá, todo es texto.

2 De la frase a las palabras que importan

Segmentación en oraciones y preprocesamiento con spaCy

Una frase tiene muchas palabras que **no aportan tema**: artículos, preposiciones, muletillas ("eh", "bueno", "digamos"). Si las dejáramos, los tópicos se llenarían de ruido. Por eso, con `spaCy` hacemos tres cosas a la vez:

- **Segmentar** el discurso en oraciones — cada oración es un "documento" para el modelo.
- **Lematizar** — llevar cada palabra a su forma base: `jugadores` → `jugador`, `entendieron` → `entender`. Así "juega", "jugaron" y "jugando" cuentan como lo mismo.
- **Filtrar** — quedarnos sólo con palabras de contenido (sustantivos, verbos, adjetivos y nombres propios) y descartar stopwords y muletillas.

PALABRA ORIGINAL	LEMA	CATEGORÍA (POS)	¿SE CONSERVA?
Estoy	estar	auxiliar	— se descarta
muy	mucho	adverbio	— se descarta
contento	contento	adjetivo	✓ se conserva
con / el / los	—	func. (prep/det)	— se descarta
equipo	equipo	sustantivo	✓ se conserva
jugadores	jugador	sustantivo	✓ se conserva
entendieron	entender	verbo	✓ se conserva
partido	partido	sustantivo	✓ se conserva
dieron / todo / en	dar / ...	verbo aux. / func.	— se descarta
cancha	cancha	sustantivo	✓ se conserva

Figura 2. Preprocesamiento real ejecutado con el código del proyecto (`preprocess.content_tokens`). De 17 palabras quedan 6 lemas de contenido. Esta es la "bolsa de palabras" que representa la frase.

RESULTADO DEL PASO 2

Estoy muy **contento** ~~con el~~ **equipo** **jugador** **entender** **partido** ~~dieron~~ **cancha**

La frase ahora es: [contento, equipo, jugador, entender, partido, cancha]

3 De las palabras a los temas (el corazón del trabajo)

Topic modeling con BERTopic: significado → geometría → grupos

Acá ocurre la magia y es donde se ve el entendimiento del tema. La idea central de **BERTopic** es: si convertimos cada oración en un punto del espacio de forma que *oraciones con significado parecido queden cerca*, entonces **los temas son simplemente las zonas densas de ese espacio**. Tres sub-pasos:

3.1 — Embeddings: convertir significado en un vector

Un modelo de lenguaje (Sentence-BERT, `paraphrase-multilingual-MiniLM-L12-v2`) transforma cada oración en un vector de **384 números**. No son palabras: son coordenadas de significado. Dos frases que quieren decir lo mismo —aunque usen palabras distintas— caen en vectores cercanos. Eso es lo que LDA, que sólo cuenta palabras, no puede hacer.

NUESTRO EJEMPLO

[contento, equipo, jugador, entender, partido, cancha] → [0.12, -0.04, 0.31, ... , 0.08] (384 dimensiones)

3.2 — UMAP: bajar la dimensión para poder agrupar

384 dimensiones son demasiadas para encontrar grupos de forma fiable (el "maldito problema de la dimensionalidad"). **UMAP** reduce esos 384 números a unos pocos, conservando la estructura de vecindad: lo que estaba cerca, sigue cerca. Es como aplanar un mapa sin romper qué ciudades son vecinas.

3.3 — Clustering: las zonas densas son los tópicos

Sobre ese espacio reducido buscamos **grupos de puntos**. Con corpus grandes usamos **HDBSCAN**, que descubre solo cuántos grupos hay; con corpus chicos —como nuestras conferencias— fijamos el número con **K-means**, más estable. Cada grupo es un tópico.

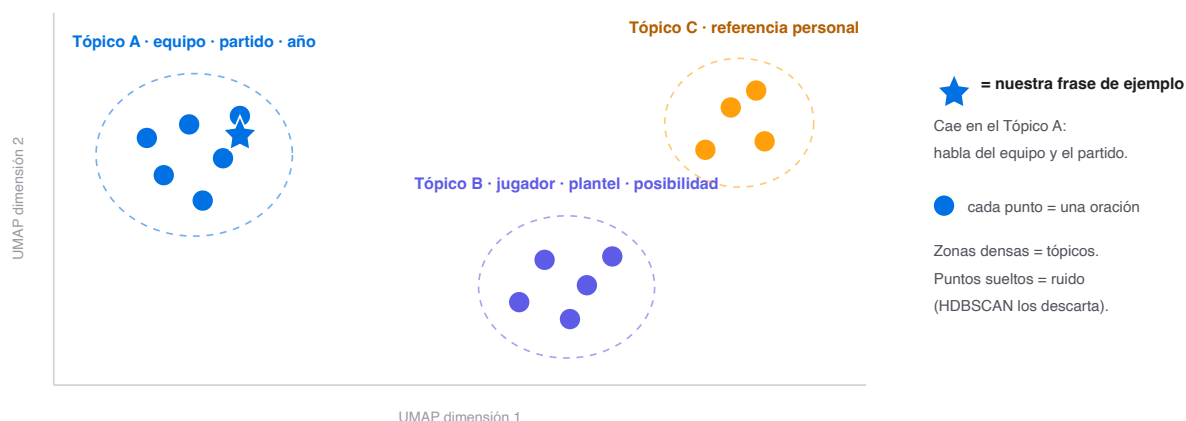


Figura 3. El espacio de significado de la conferencia de Gallardo tras embeddings + UMAP, con los tres tópicos reales que detectó el modelo. Nuestra frase de ejemplo (★) cae en el grupo del *equipo y el partido*. El modelo no recibió ninguna etiqueta: los temas emergen de la geometría.

3.4 — c-TF-IDF: ponerle nombre a cada tópico

Ya tenemos grupos, pero ¿cómo sabemos de qué habla cada uno? BERTopic junta todas las oraciones de un grupo en un solo "documento gigante" y busca las palabras que son **frecuentes en ese grupo pero raras en los demás** (eso es c-TF-IDF). Esas palabras son la etiqueta del tópico.

TÓPICO A — EN PALABRAS DEL MODELO

equipo · año · partido ← estas son las palabras que mejor distinguen al grupo donde cayó nuestra frase. Coincide con la intuición humana: la frase hablaba del equipo y del partido.

4 El baseline: LDA, otra forma de pensar los tópicos

Un modelo probabilístico clásico, para comparar contra algo sólido

Para no quedarnos con un solo método, comparamos contra **LDA**, el modelo clásico. LDA parte de una idea distinta y elegante: imagina que cada conferencia se "escribió" mezclando temas en cierta proporción (70 % táctica, 20 % rival, 10 % grupo), y que cada tema es una ruleta de palabras. LDA hace ingeniería inversa: dado el texto, **estima qué mezcla de temas y qué ruletas lo habrían generado**. No usa significado, sólo co-ocurrencia de palabras — por eso es un buen contraste contra BERTopic.

Como LDA necesita que le digamos cuántos temas k buscar, probamos varios valores y elegimos el mejor con una métrica objetiva: la coherencia.

5 ¿Qué modelo es mejor? La coherencia c_v

Una nota numérica a la calidad de los tópicos

Un buen tópico es uno cuyas palabras "pegan" entre sí: si un tema dice `gol`, `arco`, `jugada`, esas palabras deberían aparecer juntas en el texto real. La coherencia c_v mide justamente eso: toma las palabras top de cada tópico y verifica **cuánto co-ocurren en el corpus comparado con lo que ocurriría por azar**. Va de 0 (palabras inconexas) a 1 (palabras muy asociadas), y correlaciona bien con el juicio humano.

$c_v \approx$ qué tan seguido co-ocurren las palabras top del tópico $\div$ lo esperable por azar

Aplicada a nuestros modelos sobre el corpus agrupado:

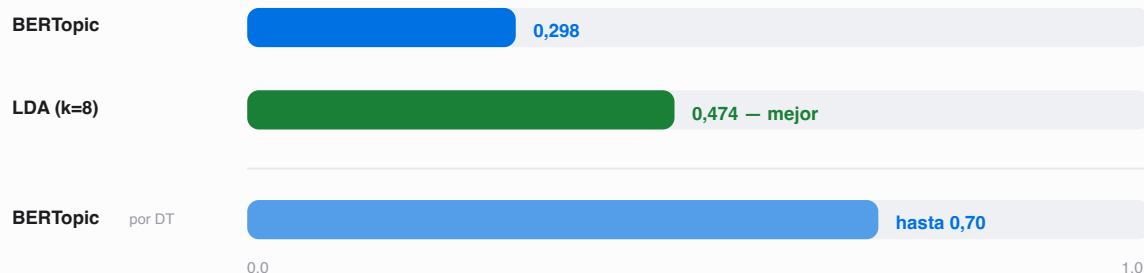


Figura 4. Coherencia c_v . Sobre el corpus agrupado, **LDA superó a BERTopic** (0,474 vs 0,298): con pocas oraciones, el clustering de BERTopic sub-segmenta. Pero **analizado por conferencia individual**, BERTopic alcanza hasta 0,70 y produce tópicos más legibles. Reportamos ambos resultados de forma transparente.

LA DECISIÓN, RESPALDADA POR DATOS

Usamos BERTopic como motor de la aplicación (tópicos legibles por conferencia) y conservamos LDA como baseline cuantitativo. No elegimos "a ojo": la métrica guió la decisión y reportamos incluso donde el baseline gana.

6 Del tópico al perfil comunicacional

La capa de aplicación: para qué sirve todo esto

Con cada oración asignada a un tópico, contamos cuánto habla el DT de cada tema y con qué tono (sentimiento, vía `pysentimiento`). Eso se resume en un radar de seis dimensiones: la **huella comunicacional** del entrenador.

Nuestra frase contribuye al perfil de Gallardo:



Figura 5. El perfil de Gallardo (escala 0–100). Una frase positiva centrada en el equipo, como la del ejemplo, empuja hacia arriba "Positividad" y "Cohesión / grupo" — exactamente el perfil que el sistema detecta para él tras un triunfo.

EL RECORRIDO COMPLETO, EN UNA LÍNEA

Audio → *"Estoy muy contento con el equipo..."* → [contento, equipo, jugador, partido, cancha] → vector de 384 dim. → Tópico A (**equipo-partido**) → suma a Positividad y Cohesión del perfil de Gallardo. **Eso es topic modeling de punta a punta.**

Todos los datos de esta nota provienen del proyecto: la frase y los tópicos son reales, el preprocesamiento se ejecutó con `backend/app/preprocess.py` y las métricas con `notebook/analisis_topic_modeling.ipynb`. Las figuras son esquemáticas pero fieles a la salida del modelo.